



CS-317

Operating Systems

LECTURE 18



Concurrency

BOOK 1 CHAPTER 5



In the last lecture...

- ▶ We explored the readers / writers problem and its various solutions.
- ▶ We realized that both the semaphore approach and the message passing approach are equally effective in solving classical synchronization problems.
- ▶ We learned how to do dry runs of long parallelly executing processes.



Coding for Concurrency

USING SEMAPHORES TO SYNCHRONIZE AND
ENFORCE MUTUAL EXCLUSION



The chocolate factory

- ▶ A chocolate factory has two types of workers.
 - ▶ Worker A makes heart shaped chocolates.
 - ▶ Worker B makes chocolates with almonds.
- ▶ Both workers start work when the manager has provided chocolate and a new box.
- ▶ Both workers put the finished candy in a box such that each box of candy has 6 heart shaped chocolates and 6 chocolate sweets with almonds in them.
- ▶ The factory has an unlimited supply of chocolate, but it may run out of almonds.
- ▶ When there are no almonds, worker B cannot make candy, until the storekeeper brings him a fresh batch of nuts.
- ▶ The storekeeper works only when worker B asks him to and sleeps the rest of the time.
- ▶ Each worker tells the manager when he has put in his 6 pieces of candy in the box.
- ▶ After both workers have completed a box, the manager provides more chocolate and a new box.
- ▶ At any one instant, only one worker may put a candy in a box.
- ▶ Insert necessary synchronization and mutual exclusion with semaphores in the given code. Do not forget to initialize them. You may use if-else structures. If you use them, place opening and closing brackets for them correctly.



Code for the processes involved

```
void worker_A ( ) {  
    int heart = 0;  
    while(true) {  
        make_candy ( );  
        heart++;  
        put_in_box ( );  
        if(heart == 6) heart = 0; } }
```

stop when 6 candies in box)

```
void manager ( ) {  
    while(true){  
        provide_chocolate ( );  
        provide_box ( );  
    } }
```

```
void storekeeper ( ) {  
    while(true){  
        /*give_almonds_to_worker_B*/;  
    } }
```

```
void worker_B ( ) {  
    int almond = 0;  
    while(true) {  
        if(out_of_almonds)  
            /*wake_up_storekeeper */;  
        make_candy();  
        almond++;  
        put_in_box ( );  
        if(almond == 6) almond = 0; } }
```

```
void main ( ) {  
    parbegin(manager, storekeeper, worker_A, worker_B);  
}
```



Manager runs before both workers.

```
void worker_A ( ) {  
    int heart = 0;  
    while(true) {  
        semWait (m);  
        make_candy ( );  
        heart++;  
        put_in_box ( );  
        if(heart == 6) heart = 0; } }
```

```
void manager ( ) {  
    while(true){  
        provide_chocolate ( );  
        provide_box ( );  
        semSignal (m);  
        semSignal (m); } }
```

```
void storekeeper ( ) {  
    while(true){  
        /*give_almonds_to_worker_B*/;  
    } }
```

```
void worker_B ( ) {  
    int almond = 0;  
    while(true) {  
        semWait (m);  
        if(out_of_almonds)  
            /*wake_up_storekeeper */;  
        make_candy();  
        almond++;  
        put_in_box ( );  
        if(almond == 6) almond = 0; } }
```

```
semaphore m = 0;  
void main ( ) {  
    parbegin(manager, storekeeper, worker_A, worker_B);  
}
```



ME must be enforced on the box.

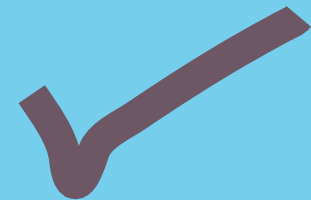
```
void worker_A ( ) {  
    int heart = 0;  
    while(true) { semWait (m);  
        make_candy ( );  
        heart++;  
        semWait (box);  
        put_in_box ( );  
        semSignal (box);  
        if(heart == 6) heart = 0; } }
```

```
void manager ( ) {  
    while(true){  
        provide_chocolate ( );  
        provide_box ( );  
        semSignal (m);  
        semSignal (m); } }
```

```
void storekeeper ( ) {  
    while(true){  
        /*give_almonds_to_worker_B*/;  
    } }
```

```
void worker_B ( ) {  
    int almond = 0;  
    while(true) { semWait (m);  
        if(out_of_almonds)  
            /*wake_up_storekeeper */;  
        make_candy();  
        almond++;  
        semWait (box);  
        put_in_box ( );  
        semSignal (box);  
        if(almond == 6) almond = 0; } }
```

```
semaphore m = 0;  
semaphore box = 1;  
void main ( ) {  
    parbegin(manager, storekeeper, worker_A, worker_B);  
}
```



Worker B wakes up the storekeeper.

```
void worker_A ( ) {  
    int heart = 0;  
    while(true) { semWait (m);  
        make_candy ( );  
        heart++;  
        semWait (box);  
        put_in_box ( );  
        semSignal (box);  
        if(heart == 6) heart = 0; } }
```

```
void manager ( ) {  
    while(true){  
        provide_chocolate ( );  
        provide_box ( );  
        semSignal (m);  
        semSignal (m); } }
```

```
void storekeeper ( ) {  
    while(true){  
        semWait (s);  
        /*give_almonds_to_worker_B*/;  
        semSignal (b);  
    } }
```

```
void worker_B ( ) {  
    int almond = 0;  
    while(true) { semWait (m);  
        if(out_of_almonds)  
        { semSignal (s);  
            /*wake_up_storekeeper */;  
            semWait (b); }  
        make_candy();  
        almond++; semWait (box);  
        put_in_box ( ); semSignal (box);  
        if(almond == 6) almond = 0; } }
```

```
semaphore m = 0;  
semaphore box = 1;  
semaphore s = 0;  
semaphore b = 0;  
void main ( ) {  
    parbegin(manager, storekeeper, worker_A, worker_B);  
}
```



Workers wake up the manager.

```
void worker_A ( ) {  
    int heart = 0;  
    while(true) { semWait(m);  
        make_candy ( );  
        heart++; semWait (box);  
        put_in_box ( );  
        semSignal (box);  
        if(heart == 6) { heart = 0;  
            semSignal (d); } } }
```

```
void manager ( ) {  
    while(true){  
        semWait (d);  
        semWait (d);  
        provide_chocolate ( );  
        provide_box ( );  
        semSignal (m);  
        semSignal (m); } }
```

```
void storekeeper ( ) {  
    while(true){  
        semWait (s);  
        /*give_almonds_to_worker_B*/;  
        semSignal (b);  
    } }
```

```
void worker_B ( ) {  
    int almond = 0;  
    while(true) { semWait(m);  
        if(out_of_almonds) {semSignal (s);  
            /*wake_up_storekeeper */;  
            semWait (b) }  
        make_candy();  
        almond++; semWait (box);  
        put_in_box ( ); semSignal (box);  
        if(almond == 6) { almond = 0;  
            semSignal (d); } } }
```

```
semaphore m = 0;  
semaphore box = 1;  
semaphore s = 0; semaphore b = 0;  
semaphore d = 2; first manager allowed but after that workers wake up the manager  
void main ( ) {  
    parbegin(manager, storekeeper, worker_A, worker_B);  
}
```



Homework: Production of glass bottles

- ▶ A factory makes glass bottles.
- ▶ A customer can place an order for either 10 transparent or 10 painted bottles.
- ▶ There are glass blowers, painters and packers in the factory.
- ▶ As long as an order is not placed, no glass is blown.
- ▶ As long as a bottle is not blown, no bottle is painted.
- ▶ If the order is for transparent bottles, the painter does not work, and the bottles are passed to the packer.
- ▶ If the order is for painted bottles, the painter paints them and passes them on to the packer.
- ▶ The packer does not work as long as he does not receive bottles. The packer places 10 bottles in a crate, after which he tells the customer that the order is ready.
- ▶ The customer must not try to pick the order before it is ready.
- ▶ A new customer must not place order before the previous one picks his order.
- ▶ Insert necessary synchronization and mutual exclusion with semaphores in the given code. Do not forget to initialize them. You may use if-else structures. If you use them, place opening and closing brackets for them correctly.



Code for the processes involved

```
void blower ( ) {  
while(true) {  
blow_bottle ( );  
}}  

```

```
void painter ( ) {  
while(true) {  
paint_bottle ( );  
}}  

```

```
void customer (int i) {  
while(true) {  
/*take_input_for_type_of_order*/;  
if (transparent bottles needed)  
order = 1;  
else order = 2;  
place_order ( );  
do_other_stuff ( );  
pick_order ( ); } }  

```

```
void packer ( ) {  
int count = 0;  
while(true) {  
if(count == 10) {  
take_new_crate ( );  
count = 0; }  
put_bottle_in_crate ( );  
count++; } }  

```

```
int order;  
void main ( ) {  
parbegin (customer(1), customer(2), blower, painter, packer);  
}  

```



Classical Synchronization Problems

THE BARBERSHOP PROBLEM



Problem Statement

- ▶ A barbershop consists of a waiting room with n chairs, and the barber room containing the barber chair.
- ▶ If there are no customers to be served, the barber goes to sleep.
- ▶ If a customer enters the barbershop and all chairs are occupied, then the customer leaves the shop.
- ▶ If the barber is busy, but chairs are available, then the customer sits in one of the free chairs.
- ▶ If the barber is asleep, the customer wakes up the barber.



Problem Statement



- ▶ To make the problem a little more concrete, the following information is added:
 - ▶ Customer threads should invoke a function named `getHairCut`.
 - ▶ If a customer thread arrives when the shop is full, it can invoke `balk`, which does not return.
 - ▶ Barber threads should invoke `cutHair`.
 - ▶ When the barber invokes `cutHair`, there should be exactly one thread invoking `getHairCut` concurrently.
- ▶ Write a program to coordinate the barber and the customers.



Classical Synchronization Problems

THE CIGARETTE SMOKERS' PROBLEM



Problem Statement

- ▶ Four threads are involved: an agent and three smokers.
- ▶ The smokers loop forever, first waiting for ingredients, then making and smoking cigarettes. The ingredients are tobacco, paper, and matches.
- ▶ We assume that the agent has an infinite supply of all three ingredients, and each smoker has an infinite supply of one of the ingredients; that is, one smoker has matches, another has paper, and the third has tobacco. The agent repeatedly chooses two different ingredients at random and makes them available to the smokers.
- ▶ Depending on which ingredients are chosen, the smoker with the complementary ingredient should pick up both resources and proceed. For example, if the agent puts out tobacco and paper, the smoker with the matches should pick up both ingredients, make a cigarette, and then signal the agent.



Problem Statement

- ▶ To explain the premise, the agent represents an operating system that allocates resources, and the smokers represent applications that need resources. The problem is to make sure that if resources are available that would allow one or more applications to proceed, those applications should be woken up. Conversely, we want to avoid waking an application if it cannot proceed.
- ▶ The restriction imposed on the system is you are not allowed to modify the agent code. If the agent represents an operating system, it makes sense to assume that you don't want to modify it every time a new application comes along. So the abilities of the agent remain as defined in the problem.



Complex Engineering Activity:

CLO: CLO 2

Taxonomy Level: C3

Three students can work together as a group.

- ▶ Implement any one of the classical synchronization problems discussed in this lecture, using any appropriate system and application programming tools, considering all the conditions and constraints specified in the problem statement.
- ▶ Provide at least three test cases to verify the correctness of your solution for the chosen problem.
- ▶ Submit a report containing the code, the test cases and execution snapshots.
- ▶ Make a demonstration video of the executing code, in which the test cases are input to it.



Complex Engineering Activity:

Rubric for evaluation

Criteria and scales		
Criterion 1: Quality of the code		
0	1-2	3
The code is wrong.	The code is incomplete.	The code is complete.
Criterion 2: Test cases		
0	1-2	3
Test cases have not been furnished.	Test cases have been furnished but are inadequate.	Test cases have been furnished and are adequate.
Criterion 3: Genuine effort		
0	2	
The code is copied.	The group has genuinely created the code.	



Lessons learned

- ▶ We applied our knowledge of synchronization to code concurrent problems.
- ▶ We learned about two more classical synchronization problems.



Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.

